

MPOO 2027-1 · GRUPO 6 · CLASE 4 · LUNES 7 DE SEPTIEMBRE

Tipos, variables y la radiografía de una clase

La P2 dejó claro que falta afinar la **anatomía**. Hoy le ponemos nombre a cada parte, en orden: variable, atributo, método y constructor. Después, los tipos y operadores que necesitan el miércoles en la **P3**.

1 Anatomía: variable, atributo, método, constructor

2 La radiografía con contexto

3 Tipos y operadores

4 Demo: del .java al .class

5 UML exprés

✓ **Ya está en main**

P00 (entorno y Git), **P01** y **P02** (abstracción y modelado) ya se integraron. Lo que entregaron vive en el repo del curso, no en su máquina.

→ **Hoy, lunes**

Primero la **anatomía** completa: variable, atributo, método y **constructor**. Luego la radiografía con contexto, y al final los tipos y operadores que usan el miércoles.

3 Esta semana

Miércoles 9: laboratorio, solo la **P3** (tipos de datos y expresiones con operadores). **Lunes 14:** entrega de la **Tarea 1** y seguimos con selección.

Lo de hoy no es cultura general: la mitad de la teoría de esta sesión es exactamente lo que van a teclear el miércoles.

double saldo = 1500.75;

TIPO: QUÉ CABE ADENTRO

NOMBRE: CÓMO LA LLAMO

VALOR: QUÉ TIENE AHORA

- **Declarar** es crear la cajita: `double saldo;`
- **Inicializar** es darle su primer valor: `saldo = 0;`
- **Asignar** es cambiarlo después: `saldo = 900;`
- El **tipo se elige una vez y no cambia**. Java es de tipado estático: si la declaraste `double`, `double` se queda toda su vida.

Y UNA VARIABLE PUEDE VIVIR EN TRES LUGARES

```
public class Cuenta {
    double saldo;                // ATRIBUTO: lo que el objeto sabe

    void abonar(double monto) {  // PARAMETRO: llega de fuera
        double nuevo = saldo + monto; // LOCAL: solo aquí
        saldo = nuevo;
    }
}
```

Todo atributo es una variable, pero no toda variable es atributo: las locales y los parámetros también lo son. Hoy nada más los nombramos; cuánto vive cada uno lo vemos en la segunda mitad, en **ámbito**.

Y todos los tipos caen en una de dos familias. **Primitivos:** la cajita guarda el valor mismo. **Referencias:** la cajita guarda a cuál objeto apunta, y si no apunta a ninguno vale `null`. Los ocho primitivos y las referencias los vemos en la segunda mitad; por ahora quédense con que la cajita puede guardar un valor o una dirección.

TRIVIA En `void abonar(double monto)`, ¿`monto` es atributo, local o parámetro?

responde en el chat

private double saldo = 0;

MODIFICADOR (OPCIONAL): LO NUEVO, LLEGA CON ENCAPSULAMIENTO

TIPO

NOMBRE

VALOR INICIAL (OPCIONAL)

Es el mismo despiece de la slide anterior, con una parte más. El atributo es la variable que va **en la clase**: por eso admite modificador y por eso nace con valor por defecto. Nombre en camelCase, sustantivo y sin abreviar: velocidad, no vel ni v1.

¿Y SI EL TIPO NO ES PRIMITIVO? ENTONCES EL ATRIBUTO ES UN OBJETO

```
public class Alumno {
    double promedio;    // primitivo: nace en 0.0
    String nombre;      // objeto: nace en null
    Materia favorita;   // otro objeto tuyo: null
}
```

null no es cero ni cadena vacía: es "todavía no apunta a ningún objeto". El atributo existe; lo que no existe es el objeto del otro lado. Por eso, antes de usarlo, hay que crearlo: `a.nombre = "Ana";` o `a.favorita = new Materia();`

Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "String.length()" because "<local1>.nombre" is null
Este **no** lo caza el compilador: truena al correr. Es el precio de que el atributo sea una referencia.

```
public void retirar(double monto) { ... }
```

CABECERA: TODO LO DE ANTES DE LAS LLAVES

FIRMA: NOMBRE + TIPOS DE LOS PARÁMETROS

CUERPO: LO QUE VA ENTRE LLAVES

La firma es `retirar(double)`

Y nada más. El tipo de retorno **NO** es parte de la firma en Java. Tampoco el nombre del parámetro ni los modificadores.

Por eso esto no compila

```
double area() { return ...; }  
void area() { ... } // misma firma: area()
```

error: method area() is already defined in class Circulo

```
// en la clase (la cabecera):  
void retirar(double monto) { ... }  
      ↑ PARAMETRO (formal)  
  
// en el main (la invocacion):  
cuenta.retirar(500.0);  
      ↑ ARGUMENTO (el valor real)
```

Invocación: objeto.metodo(argumentos). El objeto va primero porque el método le pertenece a él.

Valor de retorno

void significa que no devuelve nada: cambia el estado y ya. Si el método declara cualquier otro tipo, **tiene** que haber un return de ese tipo en todos los caminos.

Regla práctica para decidir: si quien llama necesita un dato de vuelta, el método **retorna**. Si solo debe cambiar el objeto, es **void**.

Mismo nombre y firma distinta se llama **sobrecarga**, y es legal. En la siguiente slide la van a ver trabajando, con los constructores.

public Coche(double velocidad, double gasolina) { ... }

EL MISMO NOMBRE DE LA CLASE, CON LA MISMA MAYÚSCULA

SIN TIPO DE RETORNO, NI SIQUIERA VOID

PARÁMETROS, COMO CUALQUIER MÉTODO

```
public class Coche {
    double velocidad;
    double gasolina;

    // firma: Coche(double, double)
    Coche(double velocidad, double gasolina) {
        this.velocidad = velocidad;
        this.gasolina = gasolina;
    }

    // firma: Coche() · otra firma = sobrecarga
    Coche() {
        this.velocidad = 0;
        this.gasolina = 40;
    }
}
```

- **Para qué existe:** para que el objeto **nazca completo**. Sin él, nace con los valores por defecto y hay que irle poniendo atributo por atributo.
- **Cuándo corre:** una sola vez, en el instante del new. No se invoca a mano.
- **Qué es this:** "este objeto". `this.velocidad` es el **atributo**; `velocidad` a secas es el **parámetro**. Se usa cuando los dos se llaman igual.
- **Sobrecarga, ahora sí:** dos constructores pueden convivir porque tienen **firmas distintas**: `Coche(double, double)` y `Coche()`.

Si **no escribes ninguno**, Java te regala uno vacío y por eso `new Coche()` funciona desde el día uno. En cuanto escribes el tuyo, ese regalo desaparece.

SIN CONSTRUCTOR: EL OBJETO NACE VACÍO Y TÚ LO LLENAS

```
Coche c = new Coche();  
c.velocidad = 30.5;  
c.gasolina = 12.0; // si la olvidas: 0.0
```

CON CONSTRUCTOR: NACE COMPLETO, EN UNA LÍNEA

```
Coche a = new Coche(30.5, 12.0);  
Coche b = new Coche(); // el otro
```

```
a: velocidad=30.5 gasolina=12.0  
b: velocidad=0.0 gasolina=40.0
```

Tres cosas que definen a un constructor: se llama **exactamente** como la clase, **no** lleva tipo de retorno (ni void), y corre **una sola vez**, en el instante del new. Después ya nadie lo vuelve a llamar.

1 · Olvidar el this

```
Coche(double velocidad) {  
    velocidad = velocidad; // a si mismo  
}
```

Compila sin una queja y el atributo se queda en **0.0**. Con `this.velocidad = velocidad`; sí funciona.

2 · Escribir uno y seguir usando el vacío

```
error: constructor Vacio in class Vacio  
cannot be applied to given types;  
required: double · found: no arguments
```

Si escribes un constructor con parámetros, el vacío ya no existe: o lo llamas con sus datos, o escribes también el vacío.

3 · Ponerle tipo de retorno

```
void Coche(double v) { ... }
```

Eso **no es** un constructor: es un método que se llama igual que la clase. Compila, nunca corre con new, y los atributos quedan en cero.

Cuatro líneas de comentario arriba de cada clase. No son adorno: es la parte que la P2 dejó ver que falta.

- **CONTEXTO:** ¿en qué programa vive este objeto?
- **QUÉ SABE Y POR QUÉ:** cada atributo, con su razón.
- **QUÉ SABE HACER Y POR QUÉ:** cada método: por qué void o por qué retorna, y por qué esos parámetros.
- **QUÉ IGNORE:** una o dos cosas del objeto real que no modelé, y por qué.

Si no pueden llenar la línea 1, no pueden llenar las otras tres. Todo sale del contexto.

```
// CONTEXTO: simulador de trafico; cada coche
//   avanza y gasta gasolina.
// QUE SABE Y POR QUE: velocidad (el simulador
//   la usa para moverlo cada segundo), gasolina
//   (en 0 se detiene y estorba).
// QUE SABE HACER Y POR QUE: acelerar(double kmh)
//   es void porque solo cambia su estado; kmh es
//   parametro porque lo decide el conductor.
// QUE IGNORE: marca y color, no cambian nada en
//   un simulador de trafico.
public class Coche { ... }
```

TRIVIA

Modelan un **Perro** para una veterinaria. Digan un atributo y una cosa que ignorarían.

responde en el chat

1 Circulo · programa de geometría para cortar tapas

```
double radio;  
double area() {  
    return Math.PI * radio * radio;  
}
```

Sabe: radio, porque de ahí sale todo. **Sabe hacer:** area() **retorna** porque quien pregunta necesita el número para decidir cuánto material comprar. **Ignoré:** color y posición: no cortan mejor la tapa.

Firma: `area()`

2 Coche · simulador de tráfico

```
double velocidad, gasolina;  
void acelerar(double kmh) { velocidad += kmh; }
```

void porque solo cambia su estado. **kmh es parámetro** porque no vive en el objeto: lo decide quien maneja, en el momento. **Ignoré:** marca y color.

Firma: `acelerar(double)`

El mismo coche, otro programa: en el sistema de una agencia de autos, **color** y **precio** sí serían atributos, y **velocidad** no le importaría a nadie. No hay un modelo correcto del coche: hay un modelo correcto **para este programa**.

3 CuentaBancaria · el software de un cajero

```
double saldo;  
void retirar(double monto) {  
    if (monto <= saldo) saldo -= monto;  
}  
double consultar() { return saldo; }
```

retirar es **void**: la regla de no sobregirarse vive **dentro** del objeto, no en quien lo llama. consultar **retorna**: la pantalla necesita el número.

Firmas: `retirar(double)` `consultar()`

4 Cancion · un reproductor de música

```
int duracionEnSegundos;  
double duracionEnMinutos() {  
    return duracionEnSegundos / 60.0;  
}
```

Retorna y no lleva parámetros: todo lo que necesita ya lo sabe el objeto. Fíjense en el `60.0`: con `60` sería división entera y una canción de 90 segundos daría 1.0 minutos.

Firma: `duracionEnMinutos()`

5 Alumno · sistema de calificaciones

```
String nombre;  
double promedio;  
  
boolean aprobo(double minima) {  
    return promedio >= minima;  
}
```

Retorna **boolean** porque quien pregunta necesita un sí o un no para decidir. Y **minima es parámetro**: la calificación mínima **no es del alumno**, es de la materia, del semestre, del reglamento. El contexto la pone.

Firma: `aprobo(double)`

¿Atributo o parámetro?

Una sola pregunta: ¿el dato le pertenece al objeto o se lo dan en el momento?

Le pertenece y lo carga siempre → **atributo**. Cambia en cada llamada o viene de fuera → **parámetro**.

Si **minima** fuera atributo, cada alumno cargaría su propia calificación mínima. Eso no es un sistema de calificaciones: eso es hacer trampa.

```
acelerar(double kmh) { ... }
```

error: invalid method declaration; return type required

Falta el tipo de retorno. Si no devuelve nada, se escribe void: no se omite.

```
void retirar(monto) { ... }
```

error: <identifier> expected

El parámetro va con su tipo: double monto. En la cabecera siempre son tipo y nombre.

```
double area() { double a = ...; }
```

error: missing return statement

Prometió un double y no lo entregó. Calcularlo no basta: hay que devolverlo.

```
void acelerar(double velocidad) { ... }
```

compila, y aun así está mal

Pide como parámetro algo que el objeto **ya es**. No es error de sintaxis: es **error de modelado**, y esos el compilador no los caza.

1

```
void retirar(double m)
void retirar(int m)
```

¿Misma firma?

2

```
double area()
void area()
```

¿Misma firma?

3

```
void acelerar(double kmh)
void acelerar(double vel)
```

¿Misma firma?

Firma = nombre + los tipos de los parámetros, en orden. Ni retorno, ni nombres de parámetros, ni modificadores.

1.2 TIPOS DE DATOS · LOS OCHO PRIMITIVOS

Primitivo: uno de los **ocho tipos que ya trae el lenguaje**. Su variable **guarda el valor mismo**, no la dirección de un objeto que vive en otro lado. Se escriben en minúscula porque **no son clases**: son palabras reservadas de Java.

- guarda el valor, no una referencia
- tamaño fijo y conocido
- no tiene métodos: nada de edad.algo()
- nunca vale null
- son ocho, ni uno más

TIPO	BITS	GUARDA	EJEMPLO
byte	8	−128 a 127	byte edad = 20;
short	16	−32 768 a 32 767	short anio = 2026;
int	32	±2 147 483 647	int alumnos = 29;
long	64	±9.2 · 10 ¹⁸	long ns = 900000L;
float	32	decimal corto	float t = 36.5f;
double	64	decimal normal	double saldo = 1500.75;
char	16	UN carácter	char letra = 'A';
boolean	1	true o false	boolean activa = true;

La regla del curso

Usen **int** para enteros y **double** para decimales, **boolean** para sí o no y **char** para un carácter. Los otros cuatro existen y hay que saber que existen; se usan cuando hay una razón (memoria, precisión, una API que los pide).

Y ASÍ SE VEN DENTRO DE UNA CLASE

```
public class Alumno {
    int numeroDeCuenta = 320123456;
    double promedio = 8.75;
    boolean inscrito = true;
    char grupo = '6';
}
```

'6' con comillas simples es **char**; "6" con dobles sería **String**, que ya no es primitivo. Y un **int** que se pasa de 2 147 483 647 no truena: da la vuelta a **−2147483648**.

```
double saldo = 1500.75;    // la variable ES el valor
Coche c = new Coche();      // la variable es la ETIQUETA
```

- **Primitivo:** la cajita guarda el dato. Copiarla copia el valor.
- **Referencia:** el objeto vive aparte, en el montón; la variable solo sabe dónde está. Dos variables pueden apuntar al **mismo** objeto.
- **String** es referencia, no primitivo: por eso su contenido se compara con `equals()`. Lo vemos a fondo más adelante.

Valores por defecto (solo atributos)	
int, byte, short, long	0
double, float	0.0
boolean	false
char	carácter vacío
cualquier referencia	null

Ojo, esto es solo para atributos

Las **locales** y los **parámetros** que acabamos de ver no tienen valor por defecto. Si usan una local sin darle valor, el compilador se planta: `variable x might not have been initialized.`

HACIA ARRIBA: AUTOMÁTICO (CABE, NO SE PIERDE NADA)

byte → short → int → long → float → double

```
int i = 5;  
double d = i;           // 5.0, sin pedir permiso  
c.acelerar(20);         // el 20 entra como 20.0
```

HACIA ABAJO: EXPLÍCITO, Y CON PÉRDIDA

```
int x = 3.9;             // ERROR de compilación  
int y = (int) 3.9;       // 3   (trunca, NO redondea)  
int z = (int) -3.9;       // -3  (corta hacia cero)
```

error: incompatible types: possible lossy conversion from double to int

El cast es ustedes firmando: "sé que se pierde algo y lo acepto". Por eso Java los obliga a escribirlo.

OPERADOR	QUÉ HACE	EJEMPLO
+ - * /	lo de siempre	3 * 4 → 12
%	residuo de la división	7 % 2 → 1
++ --	suma o resta 1	i++
+= -= *= /=	abreviatura	saldo += 100;
+ con texto	pega cadenas	"v = " + 30.5

La trampa número uno: división entera

```
7 / 2      // 3      (int entre int da int)
7.0 / 2    // 3.5    (si uno es double, todo es double)
```

Promedio de calificaciones con `int` y se les van los decimales sin un solo mensaje de error. Es el bug clásico de la P3.

LOS MISMOS OPERADORES, DENTRO DE UN MÉTODO

```
public class Boleta {
    double suma = 0;
    int materias = 0;

    double promedio() {
        return suma / materias;    // double entre int: da double
    }
    boolean esPar(int n) { return n % 2 == 0; }
}
```

Relacionales

Comparan y devuelven **boolean**.

```
> < >= <=
== !=
```

Lógicos

Combinan condiciones.

```
&& y
|| o
! no
```

ASÍ SE APLICA EN UNA CLASE

```
public class Alumno {
    double promedio = 8.75;
    int faltas = 3;

    boolean aprobo() {
        return promedio >= 6.0 && faltas < 5;
    }
}
```

= no es ==

= asigna, == pregunta. `if (x = 5)` ni siquiera compila en Java, y eso es una suerte: en C compila y arruina la tarde. Y el **corto circuito**: si `promedio >= 6.0` es falso, Java ya ni mira las faltas.

Nunca comparen double con ==

`0.1 + 0.2` imprime **0.30000000000000004**. No es un error de Java: es cómo se guardan los decimales en binario. Se compara con una tolerancia, no con ==.

```
public class Coche {  
  
    // de CLASE: una sola para todos  
    static int totalCoches;  
  
    // de INSTANCIA: una por objeto  
    double velocidad;  
  
    void acelerar(double kmh) {  
        // LOCAL: nace y muere aqui  
        double extra = kmh * 1.0;  
        velocidad = velocidad + extra;  
    }  
}
```

	LOCAL	INSTANCIA	CLASE
Dónde	dentro de un método	en la clase, sin static	en la clase, con static
Cuántas	una por llamada	una por objeto	una en total
Vive	lo que dura el método	lo que dura el objeto	todo el programa
Por defecto	ninguno	0 / false / null	0 / false / null

En Java no hay variables globales

Vienen de C buscando ese lugar y no existe. Lo más parecido es un atributo `static`, que pertenece a la clase y se llama por su nombre: `Coche.totalCoches`. Toda variable vive dentro de una clase.

El error que van a ver el miércoles

error: non-static variable velocidad cannot be referenced from a static context

Quisieron usar un atributo de instancia dentro de `main`, que es `static`. Primero se crea el objeto: `Coche c = new Coche();` y luego `c.velocidad`.

static · de la clase, no del objeto

El dato es uno solo y lo comparten todos los objetos. Se accede por el nombre de la clase.

```
static int totalCoches = 0;  
// dentro del constructor o de un metodo:  
totalCoches++;  
// desde fuera:  
Coche.totalCoches
```

final · se asigna una vez

Es una constante. Por convención va en MAYÚSCULAS con guion bajo. Si intentan reasignarla, el compilador los detiene.

```
static final double IVA = 0.16;  
final int RUEDAS = 4;
```

Los dos juntos, `static final`, son la forma normal de escribir una constante en Java: `Math.PI` es exactamente eso.

TODO JUNTO · ASÍ SE VE DENTRO DE UNA CLASE DE VERDAD

```
public class Termometro {
    static final double LIMITE_FIEBRE = 38.0; // constante
    static int medicionesTotales = 0;        // de CLASE

    double celsius = 36.5; // de INSTANCIA
    int lecturas = 0;
    // firma: medir(double) · retorno: void
    void medir(double nuevaTemp) {
        celsius = nuevaTemp;
        lecturas++;
        medicionesTotales++;
    }
    // firma: enFahrenheit() · retorno: double
    double enFahrenheit() {
        return celsius * 9 / 5.0 + 32; // 5.0, no 5
    }
    // firma: hayFiebre(double) · retorno: boolean
    boolean hayFiebre(double limite) {
        return celsius >= limite && lecturas > 0;
    }
}
```

DONDE QUEDO CADA COSA DE HOY

- **Tipos:** double para la temperatura, int para contar, boolean para el sí o no.
- **Constante:** static final en MAYÚSCULAS.
- **Ámbito:** medicionesTotales es de la clase; celsius y lecturas, de cada objeto.
- **Operadores:** aritméticos en enFahrenheit, relacional y lógico en hayFiebre. El casting va en el main.

Y ESTO IMPRIME, DE VERDAD

```
celsius inicial = 36.5
celsius = 38.7 | F = 101.66
redondeado hacia abajo = 38
hayFiebre(38.0) = true
lecturas de este termometro = 1
mediciones de TODOS = 1
```

```

public class MiObjeto {
    // TODO: renombra la clase Y el archivo (PascalCase)

    // ===== ATRIBUTOS =====
    // ej: double saldo = 0;
    //      tipo: double • nombre: saldo • inicial: 0
    // ===== METODOS =====
    // ej: // firma: retirar(double) • retorno: void
    //      // parametro: double monto
    public static void main(String[] args) {
        // 1: crea tu objeto y asigna atributos
        System.out.println("RADIOGRAFIA: MiObjeto");
        // 2: imprime cada atributo
        System.out.println("[estado inicial]");
        // 3: firma + invocacion + resultado
        System.out.println("[invocando metodos]");
        // 4: imprime los atributos otra vez
        System.out.println("[estado final]");
    }
}

```

- Arriba de la clase van las **4 líneas del modelo** que vimos hace rato.
- El main es la **tarjeta del objeto**: estado inicial, qué le hice, estado final. Con eso se ve si el objeto cambia como dijeron que iba a cambiar.

Lo llenamos ahora mismo, juntos, con la clase Coche. Lo que salga en pantalla es literalmente el ejemplo de su tarea.

COCHE.JAVA → **javac** → COCHE.CLASS → **java** → SALIDA

```
javac Coche.java    // nace Coche.class
java Coche          // SIN el .class al final
```

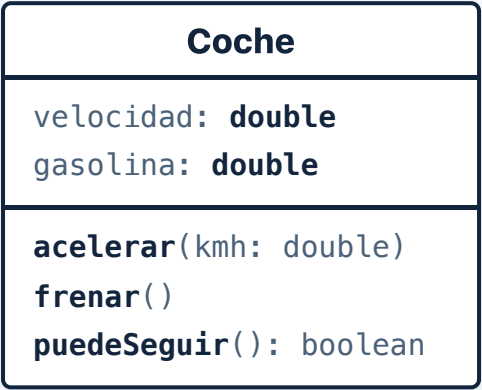
- javac traduce su texto a **bytecode**: el .class ya no es para ustedes, es para la JVM.
- Si el .class no está, no hay programa: por eso la Tarea 1 pide **subir los dos archivos**.
- Un .class compilado con JDK 21 corre en cualquier máquina con JDK 21, sin recompilar. Esa es la promesa de Java.

LA CORRIDA REAL DE HOY

```
RADIOGRAFIA: Coche
[estado inicial]
velocidad = 0.0
gasolina  = 40.0
[invocando metodos]
acelerar(double) -> void; velocidad = 30.5
acelerar(double) -> void; velocidad = 50.5
puedeSeguir()   -> boolean; true
[estado final]
velocidad = 50.5
gasolina  = 39.0
```

Tres cosas que se ven aquí: los atributos nacieron en **0.0** sin que nadie los tocara, **acelerar** no devolvió nada pero **cambió el objeto**, y al segundo llamado le pasé un **20** entero que Java convirtió solo a **20.0**.

EL COCHE DE LA DEMO



EN EL DIAGRAMA

velocidad: double	double velocidad;
puedeSeguir(): boolean	boolean puedeSeguir() {...}
acelerar(kmh: double)	void acelerar(double kmh) {...}

EN JAVA

La trampa de siempre

En UML el **tipo va después del nombre**, con dos puntos: al revés que Java. Y si el método no devuelve nada, el `: void` se puede omitir.

Tres pisos: nombre, atributos, métodos. Nada de cuerpos ni de `if`: el diagrama dice **qué**, el código dice **cómo**.

TAREA 1 · DE QUÉ OBJETO LA HAGO (Y QUÉ PRACTICA CADA UNO)

OBJETO	QUÉ SABE	QUÉ SABE HACER	QUÉ OPERADOR PRACTICA
Mochila	double pesoKg, int objetos, boolean cerrada	meter(double kg) void · estaPesada()	+= y >=
Celular	double bateria, int apps, boolean silencio	usar(double min) void · necesitaCarga()	-, < y
Cafetera	int mlAgua, int tazasServidas	servir() void · alcanzaPara(int tazas)	/ entera y %
Serie	String titulo, int vistos, int total	ver(int caps) void · porcentajeVisto()	casting y división double
Bicicleta	int cambio, double velocidad, boolean luz	subirCambio() void · vaSegura()	++, <= y &&
Taquería	int tacos, double precio, char salsa	pedir(int n) void · cambioDe(double paga)	*, - y char

Sirve cualquier objeto de su día: el refri, la ruta del camión, su rutina del gimnasio, un examen, su moto, la impresora de la facultad. La regla es que **ustedes** puedan explicar el contexto: en qué programa vive y para qué sirve.

No valen: Circulo, Coche, CuentaBancaria, Cancion, Alumno, Termometro, Videojuego ni los de las prácticas. Ya están modelados aquí; la tarea es que modelen ustedes.

TRIVIA Digan en el chat el **objeto** que van a modelar. Uno por persona, sin repetir el del de arriba.

responde en el chat

Tarea 1 · Radiografías con contexto

Entrega: lunes 14 de septiembre. Tres objetos **de su vida** (prohibidos los de las prácticas y los de hoy), desde el esqueleto del repo.

- Comentario de modelado de **4 líneas** por clase.
- 2 o más atributos y 2 o más métodos: uno **void** con parámetro, uno que retorne. Cada parte etiquetada y la **firma en comentario**.
- **Nuevo:** entre los tres objetos deben aparecer **4 tipos distintos** y **todos los operadores de hoy**: + - * / %, uno de ++ -- +=, dos relacionales, un lógico y un **casting**.
- **Un constructor** con parámetros, en **una** de las tres clases: la que no debería poder nacer vacía.
- El main completado y el **.java** **más** el **.class**. Yo ejecuto el **.class**.
- **Nuevo: un QUIZ.md** con diez preguntas de vocabulario, contestadas **con sus palabras**. Vale 20 %.

Cambia cómo se entrega: una sola rama para todo el semestre, entregas_apellido_nombre, y esta tarea vive en entregas/apellido_nombre/tareas/t01/. **Ya no se abren pull requests: solo push.**

Se califica justificación 30 %, código 30 %, que **corra** 20 % y el quiz 20 %. Instrucciones, ejemplo resuelto y menú de ideas: en el repo.

Miércoles 9 · laboratorio P3

Laboratorio de predicciones, y sí tiene entregable. El programa ya está escrito: antes de correrlo escriben qué creen que va a imprimir cada línea, luego corren y explican en qué fallaron.

Entrega: viernes 11. entregas/apellido_nombre/p03/. Se empieza en el laboratorio. Equivocarse no baja la calificación; no haber predicho, sí.

Lleguen con: el JDK 21 respondiendo (java -version), su rama del semestre y el esqueleto ya clonado.

Si hoy solo se llevan una frase: un método no es "código que hace algo", es una **firma que promete algo. El contexto decide qué prometer.**